BSc (Hons) in Computer Science & Artificial Intelligence

Course Work: Critical Review and Technical Baseline of a Machine Learning Research Paper

Reviewed Paper:

"FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning"

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024).
Publication: Proceedings of the AAAI Conference on Artificial Intelligence, 38(15), 16768-16776.
DOI: https://doi.org/10.1609/aaai.v38i15.29617

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030
- Module : Machine Learning Related Application
- Supervisor / Module Leader : Mr.Ashan Arthnayaka
- Date of Submission: Oct 15, 2026

Table of Contents

1. Introduction....3
    1.1 Why FedProto and FedTGP were selected....3
    1.2 Why the code includes many model architectures....4
2. System Architecture....6
    2.1 Main components....6
    2.2 What happens during a communication round....7
3. Libraries and how they work together....10
4. Dataset, preprocessing and limitations....11
5. Experimental setup and tuning....13
6. GPU / CUDA environment....14
7. Test run and screenshots....15
8. Results....18
    8.1 Main Results: 100 evaluations, one random seed....18
9. Comparison with the original paper....22
10. Limitations....24
    10.1 Limitations of the notebooks and experiments....24
    10.2 Limitations discussed in relation to the paper....26
11. Conclusion....27
12. References....28

FedProto Baseline Implementation & FedTGP Comparison

Phase 2 Technical Application | Option A Baseline Implementation

1. Introduction

This report describes the Phase 2 baseline implementation. The notebooks reuse components from the official FedTGP repository [2], PFLlib [3] and HtFLlib [4]. FedTGP [1] was re- run and compared with FedProto [5] on CIFAR-10 [6]. Both methods used the same saved client partitions and common local-training settings. The report describes the organization of the code, the experiments performed on the GPU in Google Colab, the results of the experiments and how these compare to the results described in the original paper.

Implementation and experiment files

The implementation notebooks are available in the GitHub repository below. Supporting experiment files are provided in the Google Drive folder.

GitHub repository: *https://github.com/ThisumiWijesinghe/FedTGP-Paper-Review*

Google Drive experiment files:

https://drive.google.com/drive/folders/1TQCzyCc24Ey5We4u4EVSRY7mWD60So7q?usp=sharing

1.1 Why FedProto and FedTGP were selected

The paper focuses on heterogeneous federated learning (HtFL), where clients may have different model architectures as well as different, unevenly distributed data. This makes standard FedAvg [7] unsuitable for this experiment: FedAvg averages model weights, which requires clients to use compatible model architectures. Here, the 20 clients use eight different networks, ranging from a small CNN to ResNet-152, so their weights cannot be directly averaged.

Other approaches also have limitations for this setting. Distillation methods such as FedMD [8] generally require a public dataset. FedKD [9] and federated mutual learning (FML) [10] introduce additional models that are trained and communicated during each round. FedGen [11] relies on a generator, while FedDistill [12] shares class-wise logits, which provide a relatively limited representation of the learned features.

Prototype-based methods offer a simpler communication strategy: clients share a feature vector for each class rather than model weights or raw data. For this implementation, there are 512 values in each prototype. With 10 classes, a client sends up to $10 \times 512 = 5{,}120$ floating-point values – 20,480 bytes (20 KB) per upload, excluding meta data. By comparison, ResNet-152 contains tens of millions of weights. This reduces the amount of information exchanged and avoids sending clients' raw data or model weights. It supports privacy, although privacy against specific attacks was not tested in this experiment.

The key reason for choosing these two methods is that they differ in how they construct global prototypes. FedProto averages uploaded class prototypes [5], while FedTGP learns them through a server-side network [1]. Using matching data and local-training settings provides a useful implementation comparison. However, prototype collection also differs between the two notebooks, so this is not an isolated test of the server update alone.

1.2 Why the code includes many model architectures

The notebooks contain more architectures than the experiment uses. These include AlexNet, BiLSTM, LSTMNet, fastText, TextCNN, HARCNN, Digit5CNN, AmazonMLP, LeNet, DNN and logistic regression, as well as the ResNet and MobileNetV2 families. They remain in the implementation for three practical reasons:

The notebooks reconstruct research-code components from the FedTGP repository [2], PFLlib [3] and HtFLlib [4]. The current FedProto client and server components come from HtFLlib. Retaining the model library makes the implementation easier to trace to its sources; notebook sections identify the original files. The contribution of this study lies in the experiment configuration, execution, evidence collection and analysis, rather than authorship of the reused algorithms.

The framework creates client models from text-based configuration strings, such as `mobilenet_v2(pretrained=False, num_classes=10)`, using Python's `eval`. Model names that can appear in a model-family configuration must therefore be defined in the notebook's scope.

The framework supports several data types, including images, text, sensor signals and reviews. Each task requires its own model architecture.

Architecture	Purpose in the framework	Used in HtFE8 CIFAR-10 run
FedAvgCNN (4-layer CNN)	Small, computationally inexpensive client model representing lower-resource devices.	Yes
GoogleNet (torchvision)	Medium-sized image model with multiple Inception branches.	Yes
MobileNetV2	Lightweight model using inverted residual blocks, suitable for resource-constrained clients.	Yes
ResNet-18, 34, 50, 101, 152 (torchvision)	Models with increasing depth and capacity, creating substantial differences between clients.	Yes
ResNet-4, 6, 8, 10 (custom)	Shallow residual networks used in other model groups, including HtFE3 and HtFE9.	No
AlexNet	Classic CNN retained for other image-model groups.	No
LeNet, DNN, logistic regression, FedAvgMLP	Small baseline models for MNIST-scale image tasks.	No
HARCNN	Model for human activity recognition using sensor data.	No
Digit5CNN, AmazonMLP	Models for Digit5 and Amazon review tasks in the KD3A benchmark.	No

LSTMNet, BiLSTM with attention, fastText, TextCNN	Text-classification models for News and Shakespeare datasets.	No

For the CIFAR-10 experiment, model group HtFE8 uses eight architectures: FedAvgCNN, GoogleNet, MobileNetV2, and ResNet-18, ResNet-34, ResNet-50, ResNet-101 and ResNet-152. The other model classes are not trained in this run; they are retained so the reconstructed code remains close to the source implementation and can support other model groups and datasets.

2. System Architecture

2.1 Main components

Each notebook contains 38 numbered sections (37 implementation sections plus the Apache licence section) and approximately 80 cells. For clarity, the implementation is organized into six layers.

Layer	Main Components	Purpose
1. Data	`read_data`, `read_client_data`, CIFAR-10 partition generator	Loads each client's `.npz` train/test files, converts data to tensors and creates the non-IID partitions.
2. Model library	ResNet blocks, MobileNetV2, AlexNet, FedAvgCNN, text models, `BaseHeadSplit`	Builds each client model and separates it into a feature extractor (base) and classifier (head).
3. Clients	Base `Client`, `clientProto`, `clientTGP`	Handles local training, prototype calculation and local evaluation.
4. Servers	Base `Server`, FedProto, FedTGP, `Trainable_Global_Prototypes`	Selects clients, aggregates or trains prototypes, evaluates performance and saves results.
5. Utilities	`result_utils`,	Reads `.h5` results, reports

	`MemReporter`, `OutputLog`, checkpoint-loader patch	GPU memory, and records console output in a log.
6. Experiment controller	Argument parsing, `run()`, `prepare_experiment`, `preserve_results`	Sets random seeds, checks GPU availability and data integrity, runs training, and saves logs and library versions.

The `BaseHeadSplit` component enables a common feature space to be shared by clients with different backbones. At the end of every backbone, the backbone is replaced by adaptive pooling which results in a feature vector of 512 dimensions (`feature_dim = 512`). This vector is used to represent the client and is used in the shared prototype space. A new linear classification head maps the 512 dimensional vector to 10 CIFAR-10 classes. Equal dimensions make communication possible, but do not by themselves guarantee semantic alignment.

2.2 What happens during a communication round

FedProto and FedTGP follow the same general sequence in each communication round:

1. The server selects clients. The join ratio is 1.0, so all 20 clients participate in every round.

2. Each client is evaluated on its own test images. A test image is assigned to the class whose global prototype is closest to the image's feature vector.

3. One local epoch, learning rate of 0.01 and a batch size of 10 per client are used; each client trains at their local site on their own data. The loss is a mixture of cross-entropy and λ times the mean squared error (MSE) between the image features and the global prototype of the correct class, with $\lambda = 10$. Global prototypes are not available in round 0, so the clients rely only on cross entropy.

4.Every client computes the mean feature vector of every class, and sends the class prototypes to the server. Raw data and model weights remain on the client.

5. The server updates the global prototypes. This is the main point where FedProto and FedTGP differ.

Item	FedProto	FedTGP
Server update	Averages the uploaded prototypes for each class. The implementation gives each client equal weight.	Trains a prototype generator using adaptive-margin contrastive learning for 100 server epochs.
Server model	No additional server model.	Class embedding of size 10×512, followed by $Linear(512, 512) + ReLU$ and $Linear(512, 512)$.
Margin	Not used.	Minimum of 100 and the largest class-wise nearest-other-class-centre distance in the inspected code.
Client prototype collection	Collected during local training.	Collected in a separate evaluation-mode pass after training.
Additional server settings	None.	`-se` specifies server epochs and `-mart` specifies the margin threshold.

For FedTGP, the server first finds the nearest other-class prototype for each class, then takes the largest of those class-wise minimum distances as the margin. During generator training, this margin is added to the distance between a client prototype and the global prototype for its true class. Cross-entropy is then applied to the negative distances. In effect, the generated prototype must still match the correct class despite this additional distance penalty, encouraging stronger separation between classes.

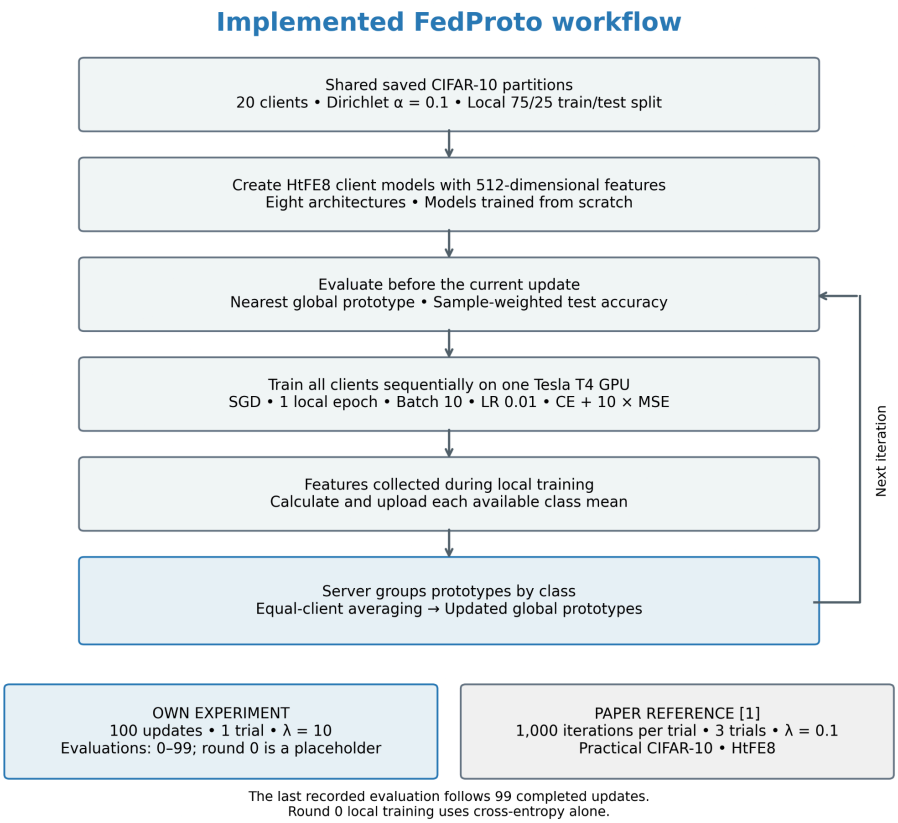


Figure 1: FedProto WorkFlow Diagram

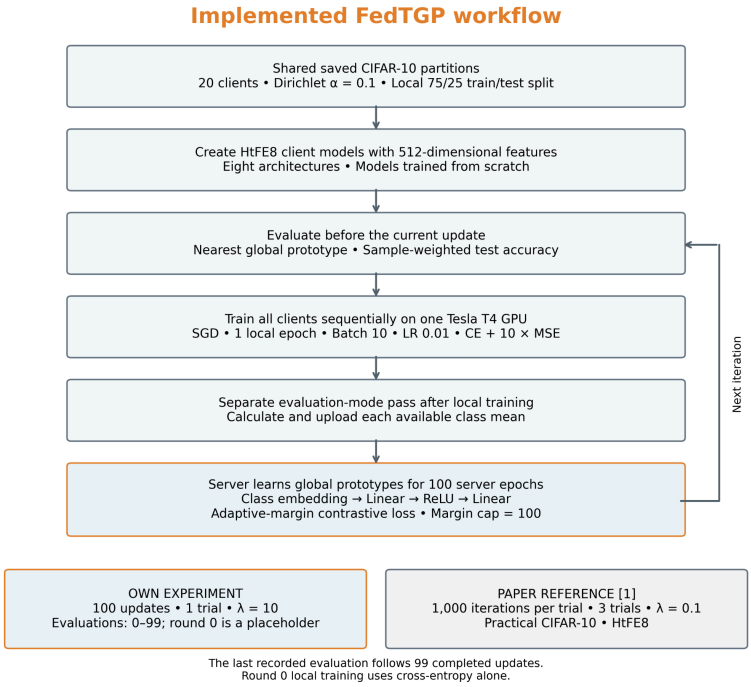


Figure 2: FedTGP WorkFlow Diagram

One implementation detail is important when interpreting the comparison. The FedProto equation in the paper weights client prototypes according to the number of samples, but the code used in this experiment averages client prototypes with equal weights.

3. Libraries and how they work together

Library	Use in this implementation	Where it is used
PyTorch(`torch`)	Tensors, automatic differentiation, SGD, CUDA execution, and checkpoint saving/loading.	Models, loss functions, the FedTGP network and device placement.
`torchvision`	CIFAR-10 download and transforms; GoogLeNet and ResNet implementations	Data preparation and `BaseHeadSplit`.
NumPy	Arrays, `.npz` client files, Dirichlet sampling and random seeds.	Data loading and partitioning.
scikit-learn	75/25 `train_test_split` per client; label binarisation and metrics imported for AUC.	Data generator and client base class.
`h5py`	Writes and reads per-round test accuracy in `.h5` files.	Server result saving and `result_utils`.
`calmsize`	Converts memory measurements into readable units.	`MemReporter`.
`ujson`	Reads dataset `config.json` files.	Partition generator.
Pillow (`PIL`)	Image handling	Partition generator.
`hashlib`, `json`, `pathlib`, `shutil`	SHA-256 data fingerprints, settings files and folder copying.	Experiment preparation and result preservation.
`argparse`, `sys`, `os`	Command-line arguments and	Experiment controller.

	path handling.	
`time`, `logging`, `warnings`	Round timing and log management.	Server loop and experiment controller.
`collections.defaultdict`	Groups feature vectors by class.	Prototype calculations.
`google.colab.drive`	Persistent storage through Google Drive.	Experiment preparation.

The components connect in a straightforward pipeline. The data layer prepares the DataLoaders, and the model library builds each client's model. Clients train locally and save their models and prototypes with `torch.save`. The server loads these files with `torch.load`, updates the global prototypes (and trains the generator for FedTGP), then saves the updated information for the next round. Between rounds, model files are stored on disk and only one client model is kept on the GPU at a time. This reduces GPU memory use, but the repeated disk reads and writes add overhead to training time. The core tensor operations and automatic differentiation are provided by PyTorch [13].

4. Dataset, preprocessing and limitations

Dataset: The experiment is conducted on CIFAR-10 [6] which consists of 60,000 color images of 32×32 size divided into 10 classes. The data generator combines the official 50,000-image training set and 10,000-image test set, normalises each channel using a mean of 0.5 and a standard deviation of 0.5, and distributes the images among 20 clients. The saved split contains 44,995 training images and 15,005 test images after rounding client-level split sizes.

Non-IID partitioning: The clients' data is divided using a Dirichlet distribution with $\alpha = 0.1$ [14]. A small α produces a strongly non-IID split: most clients have only a few dominant classes, and client dataset sizes can differ. Each client's assigned data is then randomly divided into 75% training data and 25% test data. No data augmentation or class reweighting is applied. The code does not apply missing-value imputation. File integrity and label correctness remain separate data-quality considerations.

Keeping the comparison fair: The FedTGP notebook creates the client partitions and stores a SHA-256 hash for every client file. The FedProto notebook checks that all 40 files exist and that their hashes match before starting. The message “Same FedTGP partitions verified” in the FedProto log confirms that both methods use the same client data. This helps ensure that differences in performance are not caused by different partitions. Hash agreement establishes file identity; it does not by itself rule out data leakage or incorrect labels.

The experiment has several limitations:

Each client is tested individually on its own distribution. Note that a client with many classes dominating can get a high local accuracy but have low accuracy over all 10 classes. The logged aggregate is sample weighted test accuracy: number of correct predictions / number of test examples (across all clients). It's not a mean of client accuracies weighted equally or a single model on a well-balanced global test set.

No validation set available. The “best accuracy” reported is the best accuracy that was observed over rounds on test data, and may result in better performance than it would on new test data. The final round and last 10 evaluations averages are also provided for context.

All models are trained from scratch (`pretrained=False`) from a few thousand images per client and a batch size of 10. Deeper models like ResNet-101 and ResNet-152 can be at a disadvantage in this setting.

Combining the original CIFAR-10 training and test sets before repartitioning means the experiment does not follow the standard CIFAR-10 test protocol.

Note that only one run with seed 0 was completed, and so the results of this experiment cannot be used to report run-to-run standard deviations. Repeated runs reported in the original paper are shown as means and standard deviations.

5. Experimental setup and tuning

Setting	Value	Notes
Clients / join ratio	20 / 1.0	All clients participate in every round.
Model group	HtFE8	FedAvgCNN, GoogleNet, MobileNetV2 and ResNet-18/34/50/101/152; client i receives model (i mod 8).
Local learning rate / batch size / local epochs	0.01 / 10 / 1	SGD without momentum.
Feature dimension	512	Shared prototype dimension.
λ (alignment weight)	10	Same value for both methods.
FedTGP server epochs / margin threshold	100 / 100	Server learning rate matches the local learning rate: 0.01.
Rounds	99 in the argument; 100 loop evaluations	The paper reports 1,000 rounds.
Random seed	0	Applied to Python, NumPy and PyTorch.

No hyperparameter serach was conducted. FedProto reported run time of approximately 3.8 hours on a Colab T4 GPU and FedTGP run about 5.3 hours on Colab T4 GPU, thus limiting the number of times experiments were repeated. Both methods had 100 updates and local settings. The weight applied to the alignment was $\lambda = 10$, while paper reports $\lambda = 0.1$ [1]. The experiment is therefore a low-cost, code-based comparison and not a controlled replication. Effort went into reproducibility rather than tuning, supported by the following measures:

1.Fixed random seeds for Python, NumPy and PyTorch, and to set `CUBLAS_WORKSPACE_CONFIG` before importing PyTorch.

2. Checked the client data fingerprints before the FedProto run

3. Added a guard to ensure that if an output directory already exists, an error will be raised so that an earlier run is not overwritten.

4. Saved the `settings.json` file, a list of the versions of the installed libraries (`pip freeze`), the console log to Google Drive through `OutputLog`, and the raw `.h5` file with the accuracy record to Google Drive.

5. Reported the commit number of the source code that was used as a starting point for each notebook.

The max gap recorded in the 100 FedTGP updates varied between around 2.56 and 61740.46 with 4 updates having a gap larger than 100. Therefore, the configured cap was in effect in these updates. These values are for the inspected run and cannot be used as evidence that the chosen threshold is optimal.

Two details are relevant when reading the logs. Evaluation takes place at the start of each iteration, before that iteration's training. Therefore, the evaluation labelled round 99 reflects 99 rounds of completed training. Also, round 0 reports 0.0 accuracy because global prototypes have not yet been created. This is expected behaviour in the implementation, not a runtime error.

6. GPU / CUDA environment

The experiments were performed on Google Colab, with a single Tesla T4 GPU (device ID 0, device set to CUDA). The notebook ensures that CUDA is available and prints the name of the GPU before the training starts. If the runtime does not have a GPU, the check stops the experiments early rather than allowing it to run slowly on a CPU.

Client training is sequential. The source code includes a parallel-training loop, but it is commented out, so clients are trained one after another on the same GPU. Because the client models vary considerably in size—from a four-layer CNN to ResNet-152—loading one model at a time from disk helped the experiment fit within the available GPU memory.

Across updates 1–99, the mean logged round time was approximately 137.8 seconds for FedProto and 189.1 seconds for FedTGP. The reported overall run times were 13,772.21 seconds (3.83 hours) and 18,913.04 seconds (5.25 hours). FedTGP took about 37% longer. Its 100 server

epochs and additional prototype-collection pass are plausible contributors, but they were not timed separately. This is a sequential simulation using disk-based exchange, so these timings do not measure real network latency.

At the end of the FedTGP run, the memory report showed 12 live tensors (about 1.5 KB) and 65 MB allocated. However, this measurement was taken after training, when client models had already been released. It therefore does not represent peak memory usage during training.

7. Test run and screenshots

Below are the screenshots that reflect the order of the outputs that were printed out during the run. All of them cover a different aspect of the implementation or evaluation.

```
GPU: Tesla T4
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Same FedTGP partitions verified.
```

Figure 3: Start-Up Checks in the FedProto Implementation

The Tesla T4 is detected, Google Drive is mounted, and the 20 training and 20 test partition files are checked against the saved hashes. This confirms that both algorithms use matching client partitions.

```
Algorithm: FedProto
Local batch size: 10
Local steps: 1
Local learing rate: 0.01
Total number of clients: 20
Clients join in each round: 1.0
Clients randomly join: False
Client drop rate: 0.0
Client select regarding time: False
Running times: 1
Dataset: Cifar10
Number of classes: 10
Backbone: HtFE8
Using device: cuda
Auto break: False
Global rounds: 99
```

```
Algorithm: FedTGP
Local batch size: 10
Local steps: 1
Local learing rate: 0.01
Total number of clients: 20
Clients join in each round: 1.0
Clients randomly join: False
Client drop rate: 0.0
Client select regarding time: False
Running times: 1
Dataset: Cifar10
Number of classes: 10
Backbone: HtFE8
Using device: cuda
Auto break: False
Global rounds: 99
```

Figure 4: Setting banners, FedProto(Left) and FedTGP(Right)

Both runs report the same configuration: batch size 10, learning rate 0.01, one local step, 20 clients that all join every round, CIFAR-10 with the HtFE8 model group, device cuda and 99 global rounds. Unlike other comparisons, the algorithm name is the only difference, which verifies that the comparison is made under matched settings.

```
============ Running time: 0th ============
Creating server and clients ...
FedAvgCNN(in_features=3, num_classes=args.num_classes, dim=1600)
torchvision.models.googlenet(pretrained=False, aux_logits=False, num_classes=args.num_classes)
mobilenet_v2(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet18(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet34(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet50(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet101(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet152(pretrained=False, num_classes=args.num_classes)

Join ratio / total clients: 1.0 / 20
Finished creating server and clients.
```

```
============ Running time: 0th ============
Creating server and clients ...
FedAvgCNN(in_features=3, num_classes=args.num_classes, dim=1600)
torchvision.models.googlenet(pretrained=False, aux_logits=False, num_classes=args.num_classes)
mobilenet_v2(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet18(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet34(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet50(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet101(pretrained=False, num_classes=args.num_classes)
torchvision.models.resnet152(pretrained=False, num_classes=args.num_classes)

Join ratio / total clients: 1.0 / 20
Finished creating server and clients.
Trainable_Global_Prototypes(
  (embedings): Embedding(10, 512)
  (middle): Sequential(
    (0): Sequential(
      (0): Linear(in_features=512, out_features=512, bias=True)
      (1): ReLU()
    )
  )
  (fc): Linear(in_features=512, out_features=512, bias=True)
)
```

Figure 5: Model creation list & Generator structure (a) FedProto & (b) FedTGP

This output lists the eight client backbones and, for FedTGP, the server-side generator. The generator contains a 10 × 512 class embedding, a hidden 512-unit layer with ReLU, and a 512-unit output layer. This is consistent with the server model described in the paper.

```
-------------Round number: 0-------------

Evaluate heterogeneous models
Server global_protos Not Found
Client 0: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 1: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 2: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 3: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 4: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 5: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 6: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 7: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 8: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 9: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 10: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 11: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 12: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 13: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 14: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 15: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 16: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 17: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 18: Acc: 0.0, AUC: 0
Server global_protos Not Found
Client 19: Acc: 0.0, AUC: 0
Averaged Test Accurancy: 0.0000
Averaged Test AUC: 0.0000
Std Test Accurancy: 0.0000
Std Test AUC: 0.0000
```

```
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
Server global_protos Not Found
class-wise minimum distance tensor([1.9591e+01, 1.4171e+04, 2.0804e+03, 3.4972e+03, 2.4601e+03, 7.5803e+03,
        8.2767e+03, 2.0804e+03, 1.9591e+01, 6.1740e+04], device='cuda:0')
min_gap tensor(19.5912, device='cuda:0')
max_gap tensor(61740.4648, device='cuda:0')
Server loss: 102.00070190429688
```

Figure 6: (a) Fed Proto round 0 evaluation & (b) FedTGP console after round 0

Panel (b) of Figure 6 shows what FedTGP prints after its first round. The client models have only had one epoch of training, so their prototypes are still unreliable: the class-wise minimum distances range from about 19.6 to 61,740, giving max_gap = 61,740.46 and min_gap = 19.59. Because the margin is the smaller of max_gap and the threshold of 100, the cap applies in this round, and the server loss is large (102.0). In later rounds the gaps fell to single digits (for example max_gap = 7.81 at round 99), and the cap was needed in only four rounds (0, 19, 29 and 33). This agrees with the paper's remark that a very large margin early in training can mislead learning, which is why the margin is capped.

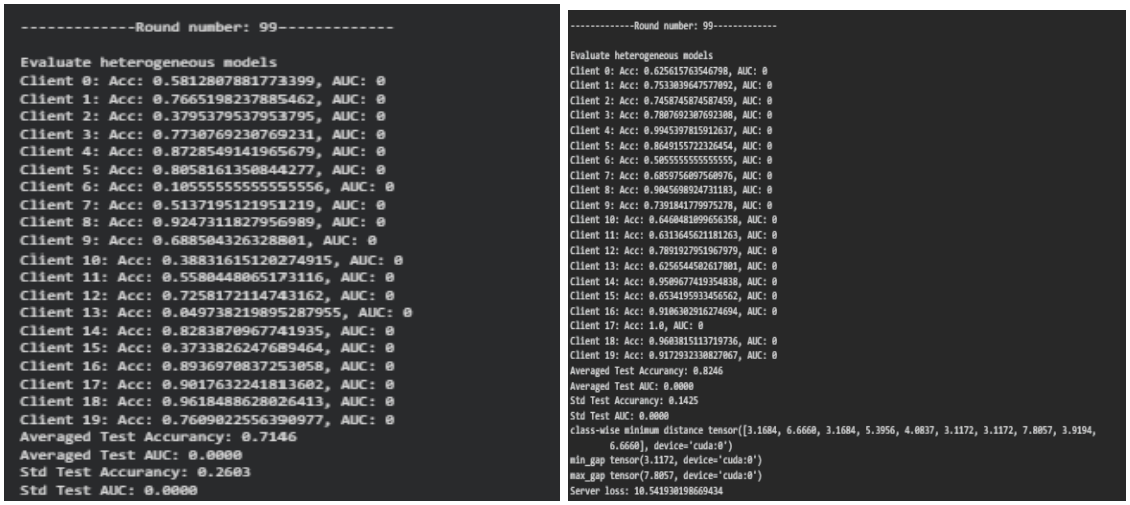


Figure 7: Round 99 evaluation, (a) FedProto & (b) FedTGP

The averaged test accuracy of FedProto is 0.7146 (71.46%) and the standard deviation of FedProto is evaluated at client level and equals 0.2603. Five of the 20 clients score below 50%, and two of them (clients 6 and 13) are near or below chance. The last value of FedTGP is 0.8246 (82.46%) with a standard deviation of 0.1425, and no client is below 50%. The FedTGP panel also reports min_gap = 3.12, max_gap = 7.81 and a server loss of 10.54 well short of the 102.0 of round 0, consistent with all of the client prototypes becoming more reliable and therefore narrowing the margin. The server loss is not constant from round to round (4.77 at round 98, 10.54 at round 99), and should not be interpreted as a trend. This is an explanation, which has not been tested independently.

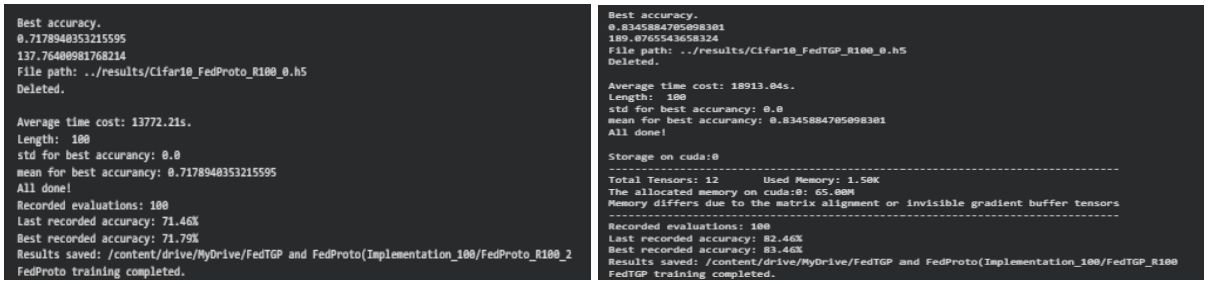


Figure 8: Final run summaries (a) FedProto & (b) FedTGP

Both runs record 100 evaluations. FedProto reaches a best accuracy of 71.79% (last 71.46%) in 13,772.21 s, and FedTgP reaches 83.46% (Last 82.46%) in 18,913.04 s. The summaries also show the Google Drive folder that holds the logs, settings and library versions, and the FedTGP panel includes the GPU memory report taken after training.

The full implementation and supporting experiment files can be accessed through the links provided in section 1.

8. Results

8.1 Main Results: 100 evaluations, one random seed

Metric	FedProto	FedTGP	Difference
Best accuracy (maximum across rounds)	71.79% (round 93)	83.46% (round 94)	+11.67 percentage points
Last-round accuracy	71.46%	82.46%	+11.00 percentage points
Mean of last 10 evaluations	71.13% (std 0.4)	75.10% (std 5.5)	+3.97 percentage points
First round to reach 50% / 60% / 70%	27 / 38 / 71	3 / 7 / 13	FedTGP reached each level sooner
Reached 80%?	No	Yes, first at round 27	FedTGP reached 80%

Standard deviation across clients at last round	0.260	0.143	Lower for FedTGP
Average seconds per round	137.8	189.1	Approximately +37% for FedTGP

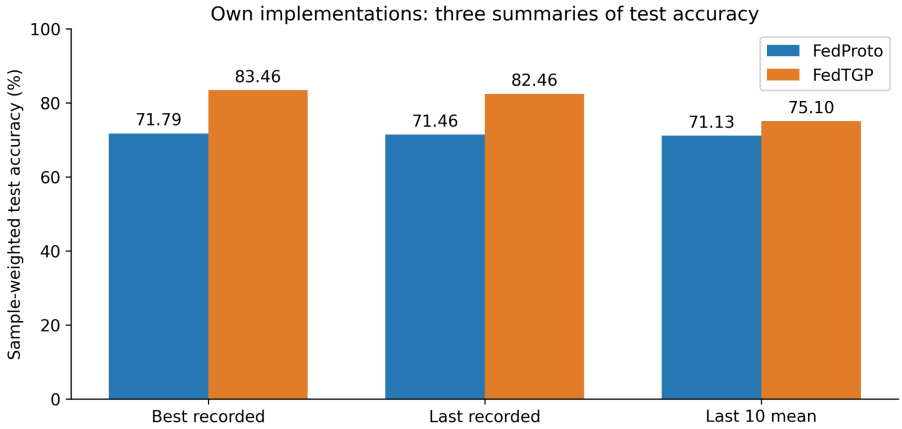


Figure 9: Best, last & last-10 mean test accuracy for FedProto & FedTGP

Figure 9 shows the three summary measures of Table 8.1 side by side. FedTGP leads on all three, but the gap narrows from about 11.7 points (best) and 11.0 points (last) to 4.0 points (last-10 mean), because FedTGP's accuracy fluctuates from round to round.

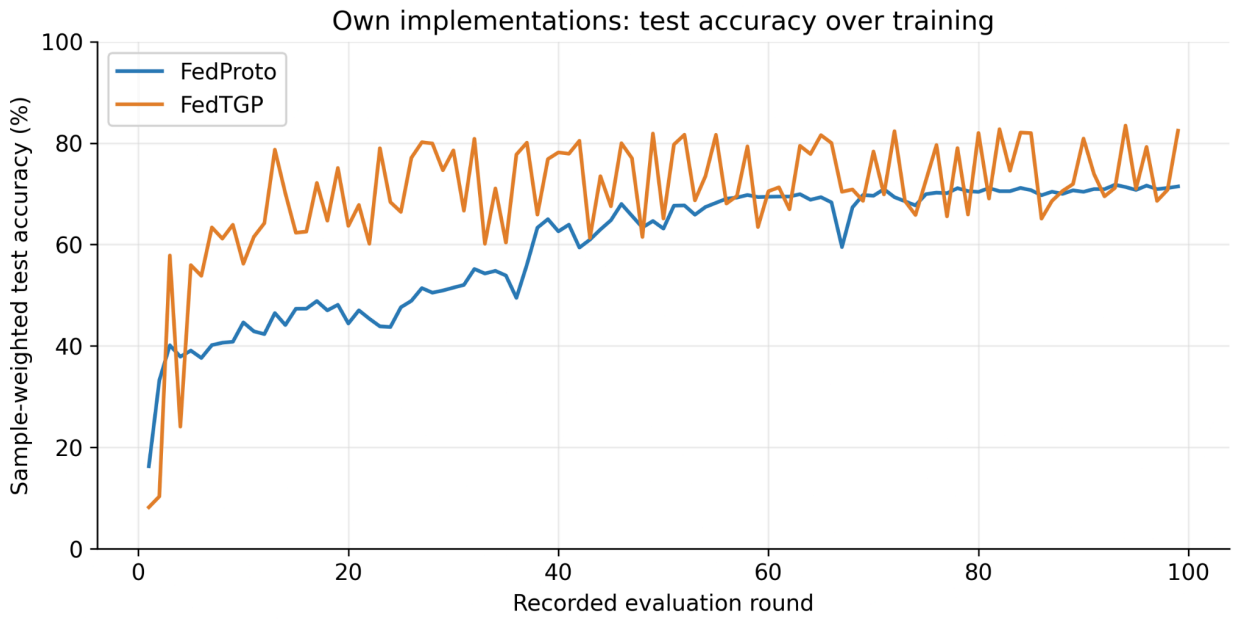


Figure 10: Sample-weighted test accuracy per evaluation round (round 0 omitted)

The curves show different learning patterns. FedProto improves gradually and is still rising at the end, from about 70.4% at round 80 to about 71.5% at round 99. FedTGP rises much earlier but less consistently, with accuracy ranging from 60% to 82% between rounds 30 and 70. This may explain why its last-10-evaluation average is only about 4 percentage points above FedProto's, although its best and last values are about 11 points higher. One possible reason is that the global prototypes change in every round while clients train with a small batch size of 10, but this was not tested.

FedTGP achieves higher accuracy on 17 of the 20 clients. The largest improvements occur among clients that perform poorly under FedProto. For example, client 13 (ResNet-50) increases from 5.0% to 62.6%; client 6 (ResNet-101) rises from 10.6% to 50.6%; client 15 (ResNet-152) improves from 37.3% to 65.3%; and client 2 (MobileNetV2) increases from 38.0% to 74.6%. Five clients score below 50% with FedProto, while none do so with FedTGP.

At the architecture level, ResNet-50, ResNet-101 and ResNet-152 clients achieve 42.8%, 46.7% and 44.4% with FedProto, compared with 74.5%, 72.8% and 67.0% with FedTGP. The smaller CNN, on the other hand, changes only a little from 80.0% to 81.4%.Because there are only two or three clients per architecture, these patterns should be treated as observations that need further

investigation, rather than firm conclusions. Architecture and data distribution vary together here, so these averages cannot isolate an architectural effect.

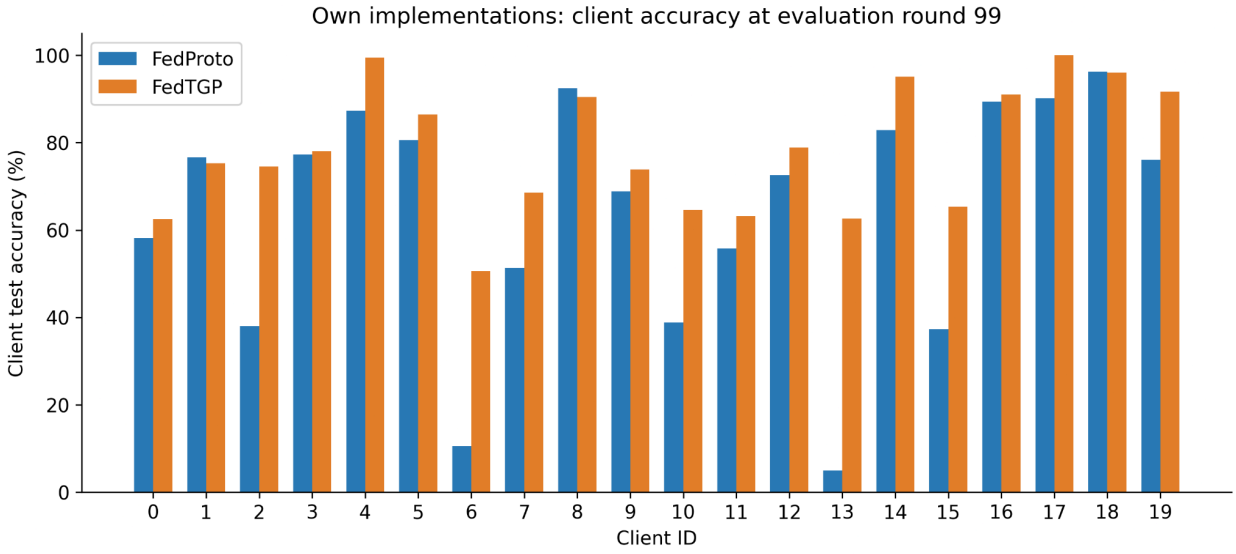


Figure 11: Per-client Accuracy

Figure 11 compares the final recorded accuracy of each client. The results come from the round-99 evaluation in the two training logs; client i uses the backbone assigned by i mod 8.

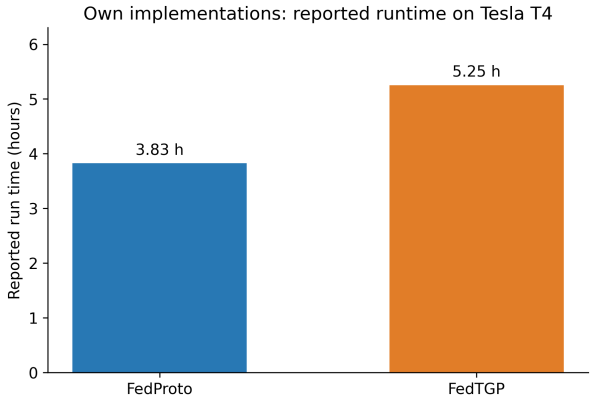


Figure 12: Runtime Bar Chart

A possible explanation, consistent with the paper's motivation, is that averaged prototypes may provide a less useful target for deeper networks that learn slowly from scratch, whereas well-separated trainable prototypes may offer a clearer target. Client 13's accuracy of 4.97% is below the 10-class chance level, which could indicate that its features were being matched to

incorrect class prototypes. However, confusion matrices were not examined, so this interpretation remains unconfirmed.

Training time also differs consistently. FedTGP adds approximately 51 seconds per round compared with FedProto (189.1 seconds versus 137.8 seconds on the T4). The per-round timing values are available in the notebook logs but are not plotted in this report.

9. Comparison with the original paper

The original paper evaluates CIFAR-10 under two settings and reports results over 1,000 rounds, including means and standard deviations from repeated runs. The Dirichlet partition used in this implementation corresponds to the paper's practical setting. [1]

Method	Paper accuracy(%), 1000 iterations; mean ± SD over 3 trails [1]	This study: best result, 100 rounds	Difference from paper
FedProto	82.07 ± 1.64	71.79%	-10.28 percentage points
FedTGP	88.15 ± 0.43	83.46%	−4.69 percentage points
FedTGP minus FedProto	+6.08 percentage points	+11.67 percentage points	—

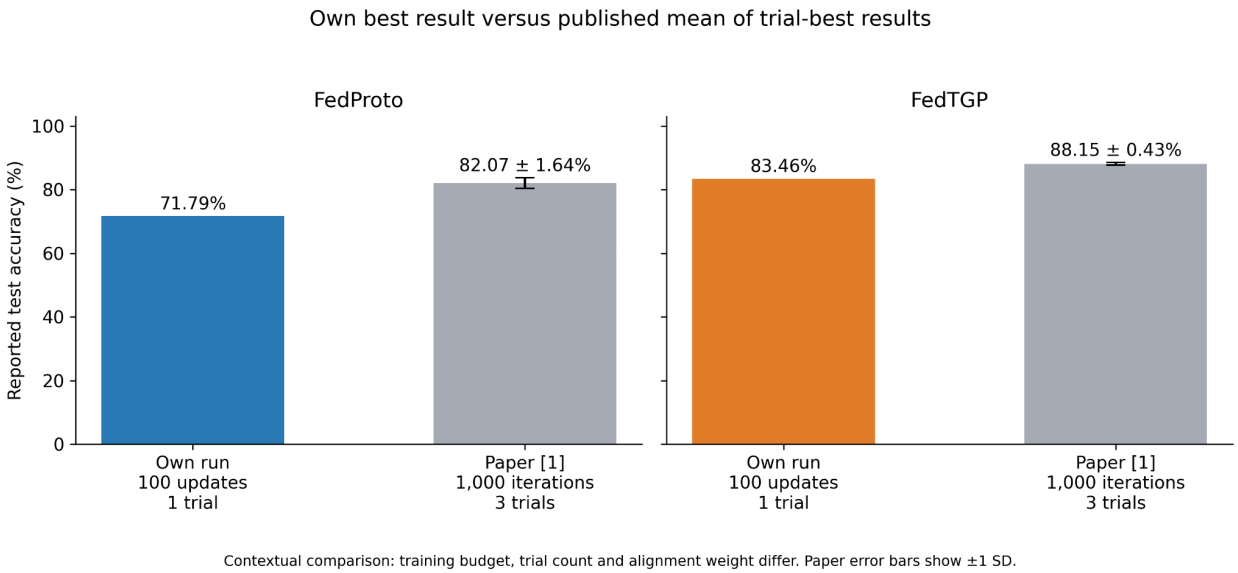


Figure 13: Own vs Paper accuracy

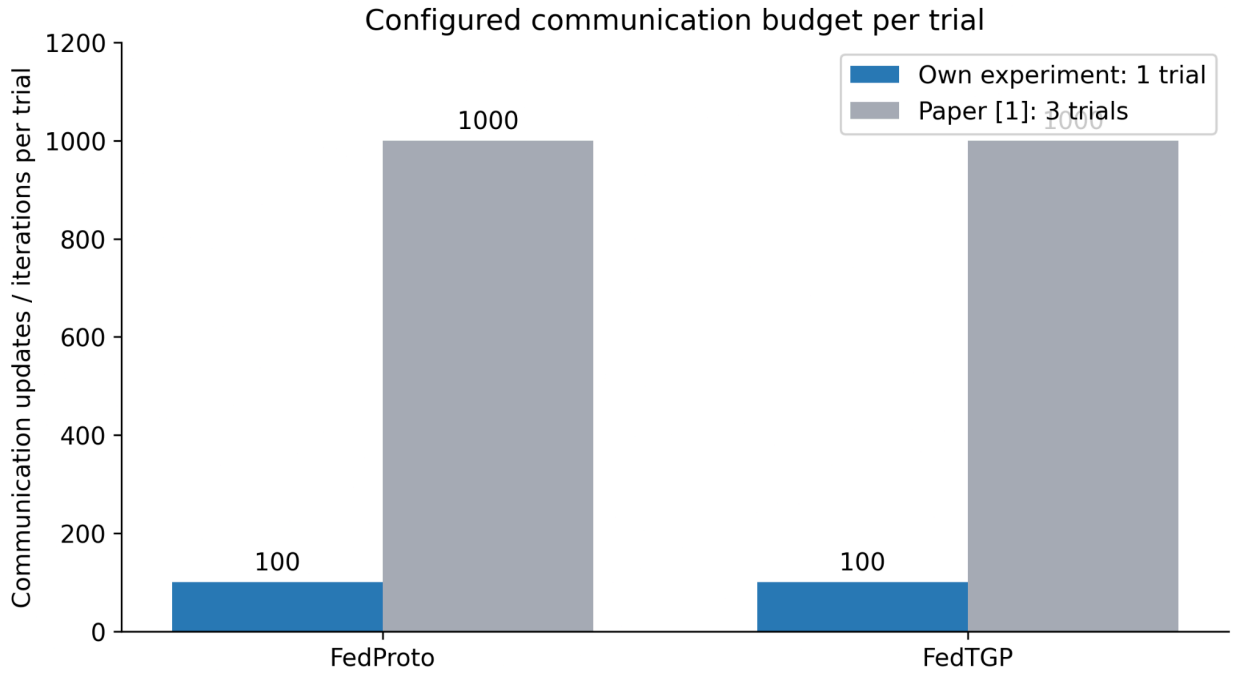


Figure 14: Communication Budget

The overall direction agrees with the original FedTGP paper [1]: FedTGP performs better than FedProto and reaches useful accuracy levels more quickly. However, the size of the difference is larger in this experiment—about 11.7 percentage points compared with the paper's 6.1. This is

mainly because the FedProto result in this shorter run is further below its published result than the FedTGP result is.

Several factors may explain why the results differ from the published values. First, FedProto was still improving at the end of the run, and only 100 rounds and one seed were used. Second, the paper reports the mean and standard deviation of three trials, whereas this study reports a single best value from one run. Third, lambda = 10 was used here, while the paper reports lambda = 0.1 [1]. For these reasons the comparison gives context only and should not be read as an estimate of the algorithmic effect in isolation.

The size of the FedTGP advantage also depends on how it is measured. The best-round values differ by 11.67 points, but the mean of the last ten evaluations (71.13% for FedProto and 75.10% for FedTGP) differs by 3.97 points. This summary reduces the dependence on a single peak, although the choice of summary is itself a reporting decision and does not replace independent validation. Overall, FedTGP reached the accuracy targets much earlier in this experiment and improved the final accuracy of most clients. Longer training and repeated seeds are needed to confirm that these differences persist.

10. Limitations

The limitations of the implementation and those of the method described in the paper are discussed separately because they affect different parts of the evaluation.

10.1 Limitations of the notebooks and experiments

Training duration: Both experiments use 100 rounds rather than the 1,000 rounds used in the paper. FedProto was still improving at the end, so its final accuracy may underestimate what it could achieve with longer training. The observed difference between FedTGP and FedProto may therefore be inflated.

Single run and seed: Only one seed was used, so standard deviations across independent runs and statistical significance tests cannot be reported. FedTGP's curve fluctuates by roughly 20 percentage points, making a single best-round result an unstable estimate.

Test-set selection: No validation set was used. Selecting the best accuracy from repeated evaluations on the test data can lead to optimistic estimates. Reporting last-round and last-10-evaluation averages provides additional context but does not remove this limitation.

Limited experimental scope: The study covers only CIFAR-10, one Dirichlet split ($\alpha = 0.1$), 20 clients, model group HtFE8 and full client participation. Other datasets, the pathological split, partial client participation and larger client populations were not tested, so the generalisability of the findings remains uncertain.

No hyperparameter search or ablation study: The executed settings were held fixed, including $\lambda = 10$ and a margin cap of 100. The cap was active in four updates, but alternative thresholds were not tested. The paper reports $\lambda = 0.1$ [1], so this difference must be retained when comparing results.

Limited metrics: Accuracy is the only usable evaluation metric in this report. AUC is printed as 0 for both methods, and per-class precision, recall and confusion matrices were not generated. Communication cost was estimated from prototype size rather than measured directly, and no privacy attack was conducted.

Training conditions: Models were trained from scratch, with batch size 10 and no augmentation. This may disadvantage the deepest networks, especially ResNet-101 and ResNet-152. Sequential training on a single Tesla T4 also limits throughput; each run took approximately 3.8–5.2 hours, making repeated runs more difficult.

Interpretation of logs: Evaluation occurs before each round's training, so the final evaluation reflects 99 completed training rounds. The 0% result in round 0 is expected because global prototypes are not yet available. Explanations of individual client behaviour—for example,

possible feature collapse for client 13—remain hypotheses because confusion matrices were not inspected and each architecture is represented by only two or three clients.

10.2 Limitations discussed in relation to the paper

Computational overhead: The FedTGP implementation trains its server generator for 100 epochs per update and performs an additional client pass to collect prototypes. Its measured mean round time was about 37% higher in this experiment. These are plausible sources of overhead, but separate profiling would be needed to attribute the increase to particular operations.

Dependence on the adaptive margin: The margin is derived from the largest class-wise minimum gap among client prototypes and is capped by a threshold. If the client features are poor early in training, the margin may also be unreliable. The paper notes that an excessively large early margin can misdirect learning, and the threshold introduces another parameter to configure.

Stability: Observed FedTGP curve varies from round to round. On the practical CIFAR-10 standard deviation reported in the paper is 0.43 percentage points, which is the cross-trial variation of the selected results [1]. It does not measure variation between successive rounds, and should not be compared with the two quantities as stability measures.

Evaluation protocol: Each client is tested on examples drawn from its own data distribution, as in personalized federated learning. The reported accuracy therefore reflects the performance of personalized clients and not performance on a balanced global test set. The experiments in the paper also concern image classification only.

Evidence for the claims: The paper supports its prototype-separation argument with comparisons and ablations [1]. This coursework did not independently test convergence guarantees or resistance to privacy attacks. Sharing prototypes avoids transmitting raw images and full model weights, but does not by itself establish protection against inversion or membership-inference attacks.

Limitations of prototypes: A single mean vector for each class cannot represent all possible visual variations within a class. Prototypes can also be noisy when a client has very few samples of a class. These limitations apply to FedProto as well as FedTGP.

11. Conclusion

FedProto and FedTGP were re-trained with the same client partitions that are verified to be hash-identical, using a Google Colab T4 GPU. The experiment consisted of 20 clients, a Dirichlet non-IID split with $\alpha = 0.1$, eight different model architectures and 100 evaluations indexed 0-99. On the test sets, FedTGP outperforms FedProto with a best sample-weighted test accuracy of 83.46%, and a test accuracy of 82.46% in the final round, while FedProto obtains 71.79% and 71.46% respectively. FedTGP reached 70% accuracy by round 13, whereas FedProto needed until round 71. FedTGP also enhanced the performance of clients that were less than 50% accurate when using FedProto. The main argument of the paper is that using trainable global prototypes can facilitate heterogeneous federated learning, which is supported by these observations.

Several points should be kept in mind when interpreting these results. The experiments were ten times shorter than those reported in the paper, so the absolute values, especially for FedProto, may be limited by insufficient training. FedTGP's curve was also somewhat unstable, and its best-round result may be higher than its average, so the advantage over the last 10 evaluations was closer to four percentage points. In addition, FedTGP needed about 37% more time per round on the available hardware.

Finally, the different alignment weight ($\lambda = 10$ here, 0.1 in the paper) and the single saved partition prevent a direct comparison with the published results.

Future work should run the full 1,000 rounds with several random seeds, record confusion matrices for the poorest-performing clients, and test a lower server learning rate to see whether it reduces the fluctuations in FedTGP accuracy. These steps would give a firmer basis for judging the stability and size of the performance difference.

12. References

[1] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 15, pp. 16768–16776, Mar. 2024, doi: 10.1609/aaai.v38i15.29617.

[2] TsingZ0, *TsingZ0/FedTGP*. (Jul. 27, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/FedTGP

[3] TsingZ0, *TsingZ0/PFLlib*. (Sep. 12, 2026). Python. Accessed: Sep. 14, 2026. [Online]. Available: https://github.com/TsingZ0/PFLlib

[4] TsingZ0, *TsingZ0/HtFLlib*. (Sep. 08, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/HtFLlib

[5] Y. Tan *et al.*, "FedProto: Federated Prototype Learning across Heterogeneous Clients," *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, pp. 8432–8440, Jun. 2022, doi: 10.1609/aaai.v36i8.20819.

[6] "CIFAR-10 and CIFAR-100 datasets." Accessed: Sep. 14, 2026. [Online]. Available: https://cave.cs.toronto.edu/kriz/cifar.html

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artif. Intell. Stat., vol. 54, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[8] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," arXiv:1910.03581, 2019, doi: 10.48550/arXiv.1910.03581.

[9] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nat. Commun.*, vol. 13, no. 1, p. 2032, Apr. 2022, doi: 10.1038/s41467-022-29763-x.

[10] T. Shen *et al.*, "Federated Mutual Learning," Sep. 17, 2020, *arXiv*: arXiv:2006.16765. doi: 10.48550/arXiv.2006.16765.

[11] Z. Zhu, J. Hong, and J. Zhou, "Data-Free Knowledge Distillation for Heterogeneous Federated Learning," 2021, doi: 10.48550/ARXIV.2105.10056.

[12] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-Efficient On-Device Machine Learning: Federated Distillation and Augmentation under Non-IID Private Data," 2018, *arXiv*. doi: 10.48550/ARXIV.1811.11479.

[13] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems, vol. 32, 2019, pp. 8024–8035.

[14] T.-M. H. Hsu, H. Qi, and M. Brown, “Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification,” Sep. 13, 2019, *arXiv*: arXiv:1909.06335. doi: 10.48550/arXiv.1909.06335.